OpenTofu: infrastructure, state and safe change

From a first plan to gradual adoption

21 October 2026 · OpenTofu 1.13.0

Contents

Infrastructure as a managed relationship **3**

1. Read OpenTofu code **3**

2. Predict the engine **13**

3. Read a small, complete system **16**

4. Review a plan as an operational proposal **21**

5. Preserve identity through change **23**

6. Own the state as an operational system **26**

7. Grow the configuration deliberately **29**

8. Adopt an existing fleet without recreating it **33**

9. Establish evidence at several levels **37**

10. Recover from partial completion **40**

11. Choose an engine and a bounded next step **44**

Worked answers **46**

A compact operational reference **51**

Reproduce the laboratories **52**

Infrastructure as a managed relationship

An infrastructure description becomes useful when it can preserve the relationship between a declared object and the real object it manages. Creating a VM is the easy case. Renaming its configuration without replacing it, bringing an existing VM under management, and recovering after half a change succeeds require a more precise model.

OpenTofu makes that relationship explicit through configuration, providers and state. The same model supports a small service and a gradual transition across an existing fleet. Its limits matter too: an API without the necessary management operations cannot be repaired by adding HCL, and two tools writing the same property do not acquire a coherent ownership policy automatically.

Chapter 1 is a reading primer, to skim and return to. Chapters 2–5 develop the engine, language, plans and resource identities. Chapters 6–10 follow those objects through shared operation, adoption and failure. Chapter 11 is an optional branch about engine choice and migration. Worked answers follow the exercises.

The examples include their expected behaviour; the accompanying files support later experiments. Source links identify the documented semantics behind consequential claims.

The target is **OpenTofu 1.13.0**, with versioned 1.13 documentation checked on **21 October 2026**. The external local-file provider is pinned to **hashicorp/local 2.5.3**. These are reproducible teaching targets, not a claim that every future patch behaves identically. Experimental 1.13 linting and symbol libraries are outside the core path. The release announcement describes that release's additions.

1. Read OpenTofu code

This chapter is a phrasebook: enough to recognise each part of an OpenTofu repository, a plan and a command's output, and to know what to look up. Most items get a sentence and a small example, usually a fragment. Why things behave as they do comes in later chapters, so skim this now and return when a construct looks unfamiliar.

OpenTofu's language is **HCL** (HashiCorp Configuration Language),

shared with Terraform. Files end in `.tf`, and a directory of them is one **root module**: every file in it is read and order does not matter, with the override files described under "Files and everyday commands" as the exception. Many names keep the Terraform spelling, such as the `terraform { }` block and `TF_VAR_`.

1.1 Hello, world

Saved as `main.tf` in an empty directory:

```
output "greeting" {
  value = "Hello, world"
}
```

```
tofu init
tofu apply
```

`init` prepares the directory, and `apply` shows a plan, asks for confirmation and carries it out. The last lines of its output, once confirmed:

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

greeting = "Hello, world"
```

Even this leaves a `terraform.tfstate` file behind, which chapter 2 explains.

1.2 Blocks and arguments

Everything is a **block**: a type, zero or more quoted labels, and a body in braces. Inside, `name = value` is an **argument**, `#` starts a comment, and a nested block has no `=`.

```
resource "local_file" "motd" {          # type, then two labels
  filename = "/tmp/motd"                # argument
  content  = "welcome"

  lifecycle {                           # nested block
    prevent_destroy = true
  }
}
```

A `resource` block's labels are the **resource type** and a **local name**. The first word of the type names the provider, here `local`. Together they make the address `local_file.motd`, which is how everything else refers to it. `.tofu` files are read too and, for a file with the same stem, replace the `.tf` file; they hold OpenTofu-only features (files).

1.3 The block types

Block	Meaning
`resource "TYPE" "NAME"`	An object OpenTofu creates and manages
`data "TYPE" "NAME"`	An object it only reads; referred to as `data.TYPE.NAME`
`variable "NAME"`	An input; referred to as `var.NAME`
`locals`	Named values computed within the module; `local.NAME`
`output "NAME"`	A value the module reports
`module "NAME"`	Use another directory of `.tf` files; `module.NAME.OUTPUT`
`provider "NAME"`	Settings for a plugin that talks to an API
`terraform`	Settings for the engine: versions, providers, state backend
`moved`, `import`, `removed`	Instructions about existing objects (chapters 5 and 8)
`check`	An assertion that warns rather than stops (chapter 9)
`ephemeral "TYPE" "NAME"`	A value used during a run and not kept in state

Inside a `resource`, a `provisioner` block runs a script on or for the object after creation, and a `connection` block says how to reach it, with `self` meaning the resource itself. They mark configuration management inside OpenTofu, which chapter 2 warns about.

The unit of reuse is the **module**, not a function: you cannot define functions in HCL. Its variables are its parameters and its outputs are its results. `source` can also be a registry address or a Git URL, and the first of these looks misleadingly like a relative path:

```
module "web" {
  source = "./modules/web"
  name   = local.name
}

module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"    # registry
  version = "~> 5.0"
}

module "dns" {
  source = "git::https://example.com/dns.git?ref=v1.2.0"
}
```

1.4 Values

```
variable "region" {
  type        = string
  default     = "eu"
  description = "Where it runs"

  validation {
    condition     = contains(["eu", "us"], var.region)
    error_message = "Unknown region."
  }
}

locals {
  name  = "svc-${var.region}"            # interpolation
  ports = [80, 443]                      # list
  owner = { name = "ops", uid = 0 }      # object or map
```

```
}
```

Types. `string`, `number`, `bool`, `list(string)`, `set(string)`, `map(string)` and `object({ role = string })`, with `any` and `optional(...)` in the same notation. `null` means unset.

`sensitive = true` on a variable or output hides the value, shown as `(sensitive value)` in a plan and `<sensitive>` in `tofu output`. It remains in the state file, as chapter 6 discusses.

Setting a variable, with later ones winning: its `default`, then `TF_VAR_region` in the environment, `terraform.tfvars`, `*.auto.tfvars`, and `-var region=us` or `-var-file=prod.tfvars` on the command line. A `.tfvars` file holds only `name = value` lines.

1.5 Expressions

Form	Meaning
`"svc-${var.region}"`	Interpolate into a string
`var.region == "eu" ? "small" : "large"`	Conditional; the only branch in an expression
`[for p in local.ports : p * 2]`	Build a list
`{ for k, v in var.hosts : k => upper(v) }`	Build a map
`[for k, v in var.hosts : k if v == "web"]`	With a filter
`terraform_data.n[*].id`	**Splat**: that attribute from every `count` instance; a `for_each` resource needs `values(...)[*].id`
`join(",", local.ports)`	Call a built-in function
`path.module`	The directory holding this file

There is no statement-level `if` or loop. Functions are built in, and common ones are `length`, `merge`, `lookup`, `coalesce`, `try`, `format` and `file`, or supplied by a provider, as in `provider::terraform::encode_tfv`

`ars(...)` (functions). `tofu console` evaluates an expression, which is the way to check one.

Heredocs. `<<-EOT … EOT` is a multi-line string with its indentation removed. Inside any string, `%{ if … }` and `%{ for … }` are template directives.

```
content = <<-EOT
  name=${local.name}
EOT
```

1.6 Repetition, conditions and errors

`count` and `for_each`. These meta-arguments make one block many instances:

```
resource "terraform_data" "numbered" {
  count = 2
  input = "n${count.index}"       # numbered[0], numbered[1]
}

resource "terraform_data" "named" {
  for_each = var.hosts
  input    = "${each.key}:${each.value}"   # named["a"], ...
}
```

`count = var.enabled ? 1 : 0` is the usual way to make a whole resource conditional, and OpenTofu also accepts `lifecycle { enabled = var.enabled }`. Chapter 5 explains why the choice between the two matters.

`dynamic` repeats a nested block inside a resource, and looks like a block called `dynamic "NAME"` with `for_each` and `content { }` inside (dynamic blocks).

Errors. No exceptions exist. A failed `validation` stops at planning. `precondition` and `postcondition` inside `lifecycle` stop the run. `check` only warns. `try(a.b, "fallback")` handles a missing attribute inside an expression.

Other meta-arguments. `depends_on = [local_file.cfg]` adds an ordering that no reference implies. `provider = aws.eu` chooses a

provider configured with `alias = "eu"`, and a module call passes one on with `providers = { aws = aws.eu }`. `lifecycle { }` holds `create_before_destroy`, `prevent_destroy`, `ignore_changes`, `replace_triggered_by`, and the conditions above. In a `removed` block it holds `destroy`.

1.7 Providers and versions

```
terraform {
  required_version = ">= 1.13.0"
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}
```

`~> 2.5` allows any later 2.x, and `~> 2.5.0` only 2.5.x. `hashicorp/local` here resolves to `registry.opentofu.org/hashicorp/local`, despite the namespace. `terraform_data` is built in and needs no entry. A `backend "s3" { … }` block in the `terraform` block says where state lives (chapter 6).

1.8 Reading a plan

`tofu plan` shows what it proposes. A symbol starts each resource, and the legend at the top lists those in use:

Symbol	Meaning
`+`	Create
`-`	Destroy
`~`	Update in place
`-/+`	Destroy, then create a replacement
`+/-`	Create a replacement, then destroy
`<=`	Read a data source
`.`	Forget: drop from state, leave the object

A real plan, with its ID shortened:

```
  # terraform_data.a must be replaced
-/+ resource "terraform_data" "a" {
      ~ id               = "706696d5-..." -> (known after apply)
      ~ output           = "x" -> (known after apply)
      ~ triggers_replace = 1 -> 2
        # (1 unchanged attribute hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.
```

The line starting `#` above a resource gives the address and verdict, sometimes with a reason on the next line, such as `# (because index [1] is out of range for count)`. `-> null` means the attribute goes away. `(known after apply)` means that the value does not exist yet. A real provider marks the argument responsible for a replacement with `# forces replacement`. `No changes.` means that configuration, state and real objects agree. `Objects have changed outside of OpenTofu` introduces drift that the refresh found.

Addresses nest: `module.web.terraform_data.svc`, `terraform_data.named["a"]`, `terraform_data.numbered[0]`. Quote them in a shell, because of the brackets.

Exit codes. With `-detailed-exitcode`, `tofu plan` returns 0 for no changes, 1 for an error and 2 for changes pending. Without it, pending changes also give 0.

1.9 Look-alikes, and why both exist

`count` and `for_each`. Both repeat, but `count` identifies instances by position and `for_each` by key, so removing an item from the middle has very different effects (chapter 5).

`resource` and `data`. Both name a type, but `resource` owns the object's lifecycle and `data` only looks it up. The same real object should not be owned by two blocks.

`variable`, `locals` and `output`. A variable comes from outside, a local is computed inside, and an output goes out.

`tofu validate` and `tofu plan`. `validate` checks the configuration by itself. `plan` also compares it with state and, through the provider, with

the real objects.

`apply` and `apply PLANFILE`. `tofu apply` makes a new plan and asks, unless `-auto-approve` removes the question. `tofu plan -out=p.bin` followed by `tofu apply p.bin` carries out the saved plan without asking (chapter 4).

`terraform_data` and `null_resource`. Both hold no real object. `terraform_data` is built in, and `null_resource` needs the `null` provider and is older.

`-target` and `-replace`. `-target=ADDRESS` limits a run to one object and what it depends on, and warns that the result is incomplete. `-replace=ADDRESS` forces one object's replacement. `tofu taint` is the older spelling of the same idea.

`moved`, `import` and `removed` blocks, and `tofu state` commands. The blocks record a change to state in reviewed configuration and appear in a plan. `tofu state mv` and `tofu state rm` change state immediately and leave no record in the repository.

`tofu` and `terraform`. They share ancestry and most of the language, not a guarantee of compatibility (chapter 11).

1.10 Files and everyday commands

Path	What it is
`main.tf`, `variables.tf`, `outputs.tf`	Conventional names only; any `.tf` file works alike
`override.tf`, `*_override.tf`	The exception: merged over blocks in other files, so a small one can change a resource
`*.tf.json`	The same language written as JSON, usually generated
`versions.tf`, `providers.tf`, `backend.tf`	The same
`modules/NAME/`	A local module, itself a directory of `.tf` files
`*.tfvars`	Variable values
`.terraform.lock.hcl`	Exact provider versions and checksums; commit it

Path	What it is
`.terraform/`	Downloaded providers and modules; do not commit it
`terraform.tfstate`, `terraform.tfstate.backup`	State and its previous version, when stored locally; may hold secrets, do not commit them
`terraform.tfstate.d/`	Local state of other workspaces
`*.tftest.hcl`, `tests/`	Tests run by `tofu test` (chapter 9)

The state file is JSON, with a `resources` list recording each object's `type`, `name`, `provider` and attributes.

Command	Meaning
`tofu init`	Install providers and modules, set up the backend
`tofu fmt`, `tofu validate`	Format, and check the configuration
`tofu plan`, `tofu apply`, `tofu destroy`	Propose, carry out, remove everything
`tofu output`	Print outputs; `-json` for a program to read
`tofu state list`, `tofu state show ADDRESS`	Inspect state
`tofu show -json PLANFILE`	A saved plan as JSON
`tofu providers`	Which providers each module needs
`tofu console`, `tofu test`	Evaluate expressions, and run tests

1.11 Everyday vocabulary

Term	Meaning
Configuration	The `.tf` files: what is intended
State	The record binding addresses to real objects

Term	Meaning
Drift	A real object has changed outside OpenTofu
Refresh	Asking providers for objects' current values
Provider	The plugin that knows one API's resources
Backend	Where the state is stored, and how it is locked
Workspace	A second state under one configuration; `tofu workspace`, and `terraform.workspace` in expressions
Root module, child module	The directory you run in, and one it calls
Registry	The source of published providers and modules

2. Predict the engine

2.1 Four things that must stay distinct

OpenTofu manages objects through APIs. A provider is a plugin that knows an API's resources, schemas and operations. The engine evaluates configuration, constructs dependencies, asks providers to observe objects, and coordinates changes. Usually this runs on a workstation or CI runner; it does not require an agent installed on each VM.

Keep four pieces in view:

1. **Configuration** describes the intended managed objects and their arguments.
2. **State** binds configuration addresses to object identities and records attributes and management metadata.
3. **Real objects** exist in the provider's system: VMs, networks, DNS records, database accounts, or local files.
4. **A plan** proposes actions using configuration, state, inputs and provider observations at a particular time.

Suppose configuration contains a resource called `server.web`, while state associates it with remote object `vm-417`. The name `server.web` is an address inside the configuration; `vm-417` is an identity understood by the provider. Changing a display name need not change either identity. Changing the configuration address can be significant even if the physical machine stays the same.

State is therefore more than a disposable cache. Without the association, the engine cannot in general know that an arbitrary existing VM is the one this resource is meant to own. A list of similar-looking machines is not a reliable reconstruction of intent. The official state purpose explains its identity and metadata role.

As a mathematical sketch, planning resembles computing a change from `(configuration, bindings, observations)` rather than evaluating a pure function of configuration alone. The observations may be incomplete; provider operations can fail; other actors can mutate the world between reads and writes. A plan is a structured prediction under those conditions.

2.2 The useful SaltStack comparison

Salt's desired-state vocabulary is a useful bridge: repeated execution should reconcile managed properties rather than replay an arbitrary installation script. The analogy stops where object identity and lifecycle become central. OpenTofu may create or destroy an entire VM to satisfy a changed argument, and its state binds that VM to a resource address.

Do not translate "ensure nginx is configured" into "put an SSH script in a VM resource". A sensible ownership split might give OpenTofu the VM, network attachment and DNS, while Salt or Ansible owns packages, files and services inside the VM. You may use another split, but it needs explicit field ownership. Two reconcilers writing the same property can repeatedly undo one another's work.

OpenTofu also does not continuously reconcile merely because configuration exists. A normal CLI invocation ends. Drift detection needs a later plan, scheduled automation, or some separate mechanism. The daemon-like controller model familiar from other platforms is a different operating arrangement.

2.3 A dependency graph, not a shell script

An expression such as `network_id = network.private.id` supplies a value and establishes a dependency. The network must be available before an operation that needs its ID. Independent resources may proceed concurrently; putting blocks earlier in a file does not establish ordering. The engine's graph also orders destruction in relation to dependencies. The documented default operation parallelism is ten. See the resource graph.

Imagine two hosts that reference a shared network. After the network is created, the hosts can be created independently. "Host A appears first" does not make Host B depend on A. Conversely, serialising every resource does not create a service deployment protocol: nothing in that order necessarily drains traffic or checks application readiness.

This matters when an agent adds `depends_on` everywhere to "make it safe". An edge needs a causal reason. A dependency answers what must precede an operation, not whether the resulting service is acceptable.

2.4 Apply is not a transaction

Suppose the network and one VM are created, then creation of the second VM fails because quota is exhausted. Successful objects can remain, and state can record them. There is no general atomic rollback across those APIs. OpenTofu's version-pinned apply implementation contract explicitly allows a partially updated result when errors occur.

The next reviewed plan should start from what actually exists. It may propose creating only the missing VM. Restoring yesterday's state would conceal the newly created objects from the engine; reverting Git would change the desired future, not reverse every API call.

For the same reason, repeatability has qualifications. A second plan can be empty when the managed properties already match. It cannot prove the web application is healthy, the backup restores, or the account is the intended account. We will add those checks separately.

Exercise 1 — reconstruct the run. A network and two VMs are planned. The network and VM A succeed; VM B fails. An agent says, "The apply failed, so nothing was committed; restore the old state and

retry." What is wrong with the proposal, and what evidence would you collect first? Worked answer

Ask an agent: "Which parts of this change are ordered, and what can remain after failure?" A good answer gives the relevant references, resource addresses, API identities and a plausible intermediate state. A weak answer cites file order, promises rollback, or confuses a completed API operation with a healthy service.

3. Read a small, complete system

3.1 The running example and its boundary

Our fictional status service has two hosts with stable keys, `status_a` and `status_b`. First we generate a handover manifest containing addresses and roles. These documentation addresses are not real machines. The manifest is a contract example, not an Ansible inventory that you can use directly, and creating it does not create hosts.

Why begin with a local file? It exercises a real provider, state, external observation and repetition without assuming a hosting platform. It also exposes an important limitation: the location of execution matters. Later, built-in-resource fixtures isolate identity and failure without an external provider.

The following is the complete `examples/manifest/main.tf`. The accompanying directory also contains a generated provider lock file and two plan-only tests.

```
terraform {
  required_version = "= 1.13.0"
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "= 2.5.3"
    }
  }
}

variable "hosts" {
  type = map(object({
```

```
    address = string
    role    = string
  }))
  default = {
    status_a = { address = "192.0.2.10", role = "status" }
    status_b = { address = "192.0.2.11", role = "status" }
  }
  validation {
    condition = alltrue([
      for host in values(var.hosts) : host.role == "status"
    ])
    error_message = "This fixture accepts only the status role."
  }
}

locals {
  handover = {
    schema_version = 1
    hosts          = var.hosts
  }
}

resource "local_file" "handover" {
  filename             =
↪  "${path.module}/generated/handover.json"
  content              = jsonencode(local.handover)
  file_permission      = "0600"
  directory_permission = "0700"
}

output "handover" {
  value = local.handover
}
```

The `terraform` spelling remains intentional in OpenTofu. A provider requirement says which plugin and version are acceptable. A provider configuration, when needed, supplies settings such as region or endpoint. The `local` provider needs none here. A backend is a third concept: where the engine stores state. It is not the provider managing this file. See provider requirements.

3.2 Read the expressions before the syntax details

`var.hosts` is a map whose values are typed objects. This is a named input, not a variable that is mutated as execution proceeds. The validation rejects other roles because this particular fixture implements only a status-service handover. It is a local contract, not a claim that every fleet has one role.

`local.handover` is a derived value. It has no remote identity and does not create anything. Its two fields make schema evolution explicit: a consumer can reject an unsupported `schema_version` rather than silently misinterpreting data.

`resource "local_file" "handover"` declares a managed file with address `local_file.handover`. The first label selects the resource type, the second is this module's name for it. `jsonencode` serialises a value; it avoids hand-building JSON with quoting and comma errors. `${path.module}` interpolates the module's path into a string.

The output exposes the structured handover. Outputs resemble return values, but a root output is also recorded in state after apply. An output is not an imperative step placed after a resource by file position. In this example it refers only to local values and can already be computed during planning. The file and the output share content, but the output does not prove that the file was written. See input variables and output values.

HCL has strings, numbers, booleans, collection and structural types, and `null`. In a resource argument, `null` means leaving the argument unset; the resource schema then determines the consequence. A missing optional argument and an empty string are not interchangeable. Avoid an agent's blanket conversion of "unset" values into empty strings. See types and null.

An expression can also be *unknown until apply*. That is neither `null` nor an error. A new VM's allocated address may not exist yet. The engine can preserve information about the value's type while deferring its value. This will matter when choosing resource keys.

3.3 Find the effective inputs

A root input can have several sources. Defaults are the fallback; environment values named `TF_VAR_name` override them. Automatically loaded `terraform.tfvars`, then `terraform.tfvars.json`, then `*.auto.tfvars`/`*.auto.tfvars.json` files in lexical order take precedence. Finally, `-var` and `-var-file` options override those sources in command-line order. An ordinary `production.tfvars` file is not loaded just because it exists: pass it explicitly with `-var-file`.

Map and object inputs are replaced as whole values, not implicitly merged. A later file supplying only `status_a` for `hosts` replaces the whole earlier map; it does not patch that entry while retaining `status_b`. This can propose removing an omitted managed host. Child-module inputs instead come from their caller's module arguments. See input precedence.

3.4 Files are an organisational choice

All ordinary configuration files in one directory form a module. Splitting this example into `versions.tf`, `variables.tf`, `main.tf` and `outputs.tf` changes readability, not ordering or scope. Nested directories are not loaded automatically. The directory in which you invoke OpenTofu is the root module; it explicitly calls child modules. See files and directories.

This is a useful review shortcut. If an agent claims that moving a block into `00-network.tf` forces it to run first, its execution model is wrong. If it proposes a subdirectory merely to separate a few variables, ask whether it intends a new module and interface.

3.5 Predict the first and second run

From a disposable copy of `examples/manifest`, the routine sequence is:

```
tofu init
tofu fmt -check
tofu validate
tofu plan -out=reviewed.tfplan
tofu show reviewed.tfplan
tofu apply reviewed.tfplan
tofu plan -detailed-exitcode
```

`init` prepares the working directory and installs the selected provider. It does not create the manifest. Validation checks configuration consistency; planning then reads state and provider observations. With no prior state or file, the first plan proposes one file resource. Applying the saved plan writes the file and state. A subsequent unchanged plan should report no changes.

For this pinned provider, editing the file externally is detected as loss of the tracked content; reconciliation recreates the expected file. The engine is not merging your manual edits. On another runner where that path is absent, the file is also absent even if shared state remembers it. This is why the fixture is useful for learning and a poor model for distributing production inventory among arbitrary runners. The provider's documentation says only that the file is generated, and overwritten if it exists; the recreation described here is observed behaviour of version 2.5.3, reproduced in the lab guide.

Exercise 2 — a successful handover. The apply succeeds and the JSON contains `192.0.2.10`. Does this establish that the status service is reachable, or even that this address belongs to the right machine? What should a real handover add? Worked answer

Ask an agent: "Show how this input becomes a provider action and a consumer contract." Expect a trace through the typed map, JSON encoding, resource and output, plus consumer validation. A weak answer paraphrases block names and claims that `validate` checks the actual fleet.

4. Review a plan as an operational proposal

4.1 Establish the target before counting actions

A plan saying "one resource to change" is reassuring only after you know what it addresses. The target has several independent dimensions: root directory, selected state/backend, CLI workspace, variable values, provider configuration, and effective credentials. An apparently identical directory can reach a different account through an environment variable or credential profile.

Before a real change, ask for the engine version, source revision, selected provider versions, backend/state identifier, workspace and provider identity. Provider identity should be observed through the relevant API or equivalent trusted command: account/project, region and principal, not just a profile name in HCL. The backend can itself use a different credential chain from the resource provider.

For a personal project you may perform these checks interactively. For a shared production fleet, make the same evidence part of the run record. A human should be able to distinguish "the intended production state" from "a state file whose name happens to include prod". These are design recommendations; the engine cannot infer your organisational intention from names.

4.2 Read actions and their reasons

Typical plan notation includes `+` create, `~` update in place, `-` destroy, and a combined destroy/create marker for replacement. Read the exact legend in the output. A replacement is materially different from an update even if the input change is a single line.

For each consequential resource, examine its full address, remote identity where available, changed arguments and replacement explanation. A provider decides which changes can update an object and which require replacement. A hostname, image or disk setting may have different lifecycle consequences across providers and resource versions. "Only a tag changed in Git" does not establish the API effect.

Unknown values need interpretation. An allocated IP being unknown

for a new machine is ordinary. A security rule's scope becoming unknown after an unrelated module edit deserves investigation. Ask which dependency made it unknown and what cannot now be verified before apply. Unknown is not permission to assume the eventual value is harmless.

Ordinary planning refreshes observations; disabling refresh can conceal external change. The plan command also offers targeted planning for exceptional recovery, but a routinely targeted plan is not evidence about the whole root. Use deliberate root/state boundaries for routine independent operations. See plan modes and options.

4.3 Saved plans and the approval boundary

An unsaved plan is a preview. Running bare `tofu apply` later calculates another plan, which can differ. To apply the reviewed proposal, save it with `-out`, inspect that artifact, and pass that artifact to `apply`.

Applying a saved plan does not ask for confirmation. Supplying it is treated as approval. A CI job or an agent with permission to execute that command can therefore cross the change boundary immediately. Tie permissions to the intended workflow. See saved-plan apply.

The artifact needs provenance: source revision, inputs, engine/provider selections and intended state. Changing the source after creating a plan does not turn that existing artifact into a plan for the new source. Stored planning decisions are exactly why the workflow is reviewable, and why mixing artifacts from different runs is dangerous.

A saved plan also does not reserve remote capacity or prevent console edits. State changes can make it stale; external changes need not advance this state's revision. A long delay, unexpected intervention or changed operational conditions justifies discarding it and obtaining a fresh reviewed plan. Do not claim that it constitutes a distributed lock or a service rollout guarantee.

4.4 A plan is not an acceptance test

Consider an intended replacement of one status VM. The plan correctly creates a new VM before deleting the old one. The new VM's

API status becomes running; its application is broken. From an infrastructure perspective the API calls may have succeeded. From your perspective the rollout failed.

Acceptance must state what else matters: correct machine identity, reachable management path, expected service response, removal from and re-entry into traffic, configuration handover, preserved data, and an acceptable disruption window. Choose these according to the project. A small nonprofit website may accept a short outage; a production customer path may require staged cutover and a backout arrangement.

An agent can write the HCL and propose those checks. You retain the decision about which failures and interruptions are acceptable. "All green" without the test's subject and assertion is not adequate evidence.

Exercise 3 — the changed plan. You review a saved plan replacing a test VM. Someone then edits the HCL to point at production and runs `tofu apply reviewed.tfplan` in the original test working directory, without reinitialising it. Does that command automatically replan for production? What separate risk remains even when applying the intended test artifact? Worked answer

Ask an agent: "Which exact artifact will execute, against which identities, and what result would still count as failure?" Expect the artifact provenance, effective target evidence, replacement details and external acceptance criteria. Warning signs include only a plan summary, unexplained `-auto-approve`, or applying bare `apply` after approval of a different preview.

5. Preserve identity through change

5.1 Identity is part of the interface

Suppose you want a resource per host. You can use `count` to create numeric instances, or `for_each` to create keyed instances. The choice defines persistent addresses, not merely loop syntax.

This excerpt is deliberately provider-neutral and not a complete runnable module:

```
# An index is the identity, not the name at that index.
resource "terraform_data" "indexed" {
  count = length(var.names)
  input = var.names[count.index]
}
```

For names `["a", "b", "c"]`, the associations are `[0] → a`, `[1] → b`, `[2] → c`. Remove `b`: `[1]` now describes `c`, while `[2]` is removed. With `terraform_data`, the changed input can update in place; with another resource type it might cause replacement. The error is assuming that list elements carry identity independently of their indices, not assuming that every such edit necessarily recreates everything. See count.

For durable named machines, stable keys usually fit better:

```
resource "terraform_data" "host" {
  for_each         = var.hosts
  input            = each.value.role
  triggers_replace = each.value.revision
}
```

Here the addresses include `terraform_data.host["status_a"]` and `terraform_data.host["status_b"]`. Removing the second key need not change the first instance. The complete `examples/lifecycle` fixture defines each host as an object with a role and numeric revision.

Choose keys as deliberately as database primary keys. A customer-facing display name may change; an allocated IP may be unknown until apply. Neither is a good default identity for a durable host. `for_each` keys must be known before remote operations and cannot be sensitive. Values under known keys may contain unknown attributes. `for_each` accepts maps or sets of strings; converting a list with `toset` removes duplicates and ordering. See for_each.

5.2 A lifecycle laboratory

`terraform_data` is a real built-in resource that records values in state without provisioning a VM. Its `input` can change in place; changing `triggers_replace` forces replacement. It needs no downloaded provider. That makes it a controlled fixture for reasoning about identity; it does

not simulate cloud quotas, external drift or provider timeouts. See terraform_data.

For the supplied defaults, applying creates two IDs. Planning with `update.tfvars` changes only `status_a`'s role: one update, preserving its ID. Planning from that same initial baseline with `replace.tfvars` increments its revision: one replacement, while `status_b` stays untouched. These are alternative experiments from one baseline, not an instruction to apply both files in sequence and expect no interaction.

The exact prediction is useful. If a generated implementation changes both IDs while trying to change one host's role, ask which identity or lifecycle assumption changed. A matching resource count is insufficient: "two before, two after" can conceal replacement of both.

5.3 Refactor with an explicit relationship

Renaming `host` to `node` changes addresses. Without a recorded relationship, the engine can interpret old addresses as removed and new addresses as new resources. When the intended operation is a rename, preserve the association:

```
# Accompanies renaming the resource block host -> node.
moved {
  from = terraform_data.host
  to   = terraform_data.node
}
```

The plan should show moves without infrastructure replacement if the arguments still match. A move does not hide a real argument change. For a `count` to `for_each` transition, map each old numeric address to its intended new keyed address; do not assume an automatic inference from similar names. Preserve move history when downstream users may skip versions. See module and resource refactoring.

This gives a good rule for a long transition: separate a structural refactor from an infrastructure change when practical. First prove the new addresses still own the same objects. Then make the intended behavioural change. Smaller causal steps make unexpected replacements easier to explain.

5.4 Replacement policies have limits

Replacement normally deletes the old object before creating its successor. `create_before_destroy` reverses that order; it requires temporary capacity and names that allow coexistence. It does not establish service readiness or switch traffic. Its ordering can propagate to dependencies and be recorded in state.

`prevent_destroy` rejects planned destruction or replacement while that rule remains configured. Removing the whole resource block removes that protection. `ignore_changes` can delegate updates to selected fields to another writer, but it can also hide accidental drift. It still permits the resource to be created or destroyed. These behaviours are documented under resource behaviour.

For a database, additional provider-side deletion protection, access restrictions and tested backups may be justified. That is a layered operational design, not a magical lifecycle flag. For a stateless disposable host, replacement may be routine, but only after the service cutover requirements are defined.

Exercise 4 — a cosmetic rename. An agent changes map key `status_a` to `status_primary` for readability and adds `create_before_destroy`. It describes the result as "a harmless rename with zero downtime". Explain the two unsupported conclusions and propose the evidence needed to accept a real rename. Worked answer

Ask an agent: "What preserves each existing object's identity through this edit?" Expect old and new addresses, remote IDs, explicit moves where needed and a plan accounting for every replacement. A weak answer says that names are cosmetic or that a lifecycle flag proves availability.

6. Own the state as an operational system

6.1 Choose the unit of change

One root invocation manages a state and a graph. A single state for an entire organisation is simple to discover but couples unrelated changes, access and recovery. A state per individual resource makes every relationship an integration problem. Neither extreme follows from the

number of hosts.

Choose boundaries around meaningful ownership, permissions, lifecycle and blast radius. A shared network, a production application, and a disposable development environment may deserve different states. A small service can reasonably begin with one root. For an existing fleet, the transition and ownership boundaries matter more than an arbitrary host-count threshold.

Splitting state removes graph edges across the boundary. If application infrastructure needs a network ID from another state, a separate system must arrange "network ready before application", publish an interface, and handle breaking changes. Splitting improves independence only when that interface is stable enough to support it.

Do not split merely to eliminate an inconvenient plan. If two roots both believe they own one firewall rule, separate locks cannot make that ownership coherent.

6.2 What a backend does

A backend implements state storage and related coordination. Local state is suitable for these disposable labs. Shared operational use usually needs durable remote storage, controlled access, recoverable history and locking where supported. Backend support is built into the CLI; it is separate from resource provider configuration.

Initialisation records backend details in the working directory's `.terraform` data. That local metadata is not a copy of your HCL to distribute as source. Credentials written into backend arguments or `-backend-config` can persist in local metadata and plans. Prefer the backend's standard credential mechanism and review the runner's effective identity. See backend configuration.

Changing storage is a migration. Freeze writers, back up the current state using an authorised secure process, initialise with the intended migration operation, confirm the destination, and prove that the next plan addresses the same objects. `init -reconfigure` and a state migration are different intentions; do not let an agent choose by trial and error after an ambiguous prompt.

For an S3 backend, OpenTofu 1.13 supports native locking with `use_`

`lockfile = true`; the documentation recommends that mechanism and bucket versioning. DynamoDB locking remains supported and is not scheduled for OpenTofu deprecation. An article about Terraform's backend policy is not sufficient evidence about OpenTofu. The actual permissions and object paths must come from your chosen backend design. See the OpenTofu S3 backend.

6.3 What a lock does not do

For supporting backends, state locking coordinates writers of that state. It does not prevent a console user changing a VM, and it does not coordinate another state that owns the same VM. Do not disable locking to get past contention. Establish whether the lock belongs to a live run; force-unlock is an abandoned-lock recovery operation after that investigation. See state locking.

The simplest organisational policy is usually a single authorised writer path per state, even if several people can propose changes. For a personal project that path may be your workstation; for a shared fleet it may be CI with human approval. Read access to state is itself a privilege, so "everyone can plan" is a security decision rather than merely a convenience.

CLI workspaces select different state instances using shared configuration and backend arrangements. They can be handy for similar temporary environments. They do not create separate credential boundaries or automatically make remote object names unique. Distinct production/development permissions usually justify separate roots and backend arrangements that call shared modules. See workspace limitations.

6.4 Secrets follow the data

A password marked `sensitive` is redacted in ordinary presentation; the marking does not make normal persisted values disappear. State and plan access may reveal them. Do not put state, plans or `.terraform/` in Git. Do keep the provider lock file. Be equally deliberate about CI artifacts, debug logs and outputs copied into an agent's context. See sensitive state data.

OpenTofu has client-side state and plan encryption. They are configured separately. Encryption at rest does not stop the authorised run-

ner reading plaintext, preserve lost data, or prevent replay of an older valid snapshot. Key recovery is part of state recovery. Before enabling it operationally, prove that a replacement runner can read the data and that the key can be recovered. Migrating existing plaintext state requires the documented transition rather than assuming a new encryption setting rewrites every historical copy. See state and plan encryption.

Use narrow handover interfaces. `terraform_remote_state` makes root outputs convenient to reference, but the reader requires access to the entire state snapshot, and sensitivity markings are not propagated as a confidentiality boundary. Publishing a non-secret network ID or host manifest through an authorised configuration channel can give a consumer less access. The choice has costs: schema, publication and versioning now need an owner. See remote-state access.

Exercise 5 — isolation by name. A proposal puts development, staging and production resources into three CLI workspaces in one root, using one powerful credential. It says this guarantees isolation. Which kinds of isolation exist, and which decisions remain unresolved? Worked answer

Ask an agent: "Who can read or write this state, and what happens if the runner or keys disappear?" Expect backend and principal evidence, lock behaviour, backup/version history and a tested recovery path. Warning signs include secrets committed as tfvars, lock suppression, or "encrypted" without a key recovery design.

7. Grow the configuration deliberately

7.1 A module should hide a policy, not just some lines

The directory you have been using is already a module: the root module. A child module packages configuration behind input variables and outputs. Calling it does not give it separate state; its resources join the caller's graph and state under addresses such as `module.status.terraform_data.host["status_a"]`.

For a handful of resources with one owner, keeping a single root is of-

ten clearer. Extract a child module when there is a coherent contract worth maintaining: a standard service host, a network attachment with specific rules, or the handover schema. Repetition is evidence of possible shared policy, not proof that every repeated block needs a wrapper.

A useful interface might accept a map of stable host keys and required network IDs, then expose a map of host addresses. It should explain which details are policy and which are intentionally configurable. A module exposing every underlying provider argument adds indirection while leaving the caller to understand the whole provider anyway.

Here is an interface sketch, not a supplied runnable cloud module:

```
module "status" {
  source = "./modules/status"
  hosts  = var.status_hosts
}

output "handover" {
  value = module.status.handover
}
```

The module's implementation defines the input type and output. Its source directory is a dependency, not an automatically discovered folder. Moving existing resources into it changes addresses; include the corresponding moves and inspect the plan. Consult module composition for the distinction between composing capabilities and constructing deeply nested abstractions.

7.2 Dependencies must express the real reason

Prefer references to values actually needed by a resource. If a VM requires a subnet ID, pass that subnet ID. `depends_on` is for a dependency that exists through behaviour but is not expressed by such data: for example, an access-policy attachment that must be completed before an API operation using the associated role.

A broad module-level dependency can make many values unknown during planning and make plans unnecessarily conservative. It does not provide a health check. If a generated change adds `depends_on = [module.network]` after a mysterious replacement appears, ask whether

it is addressing a real hidden dependency or masking the underlying interface problem. See depends_on and planning consequences.

A data source reads an existing object rather than declaring ownership of its lifecycle. It is appropriate when another system owns that network or image. The read may happen during planning, or be deferred when its inputs or dependencies are not ready. A query that returns "the newest matching image" makes time an input; the next plan can change without a Git edit. That may be intended for a discovery workflow, but it is a poor unnoticed production default. See data-source behaviour.

During adoption, this distinction is particularly valuable. A host that remains externally owned can be passed as a validated input or looked up with a data source. Do not declare a managed VM resource merely to obtain its address: that declaration says the root should manage the VM.

7.3 Providers are part of the target

Configure providers in the root and pass the intended configurations to child modules. Aliases distinguish multiple configurations of one provider, such as regions or accounts. Each alias needs a clear identity; its name does not verify its credentials. A resource accidentally using the default configuration can target a different account from neighbouring resources.

An implementation should make provider selection inspectable. If a module expects an aliased configuration, its requirements must declare that expectation and its caller must supply it. Avoid modules with embedded production credentials or a hidden region decision. See provider configuration and aliases.

For an inherited repository, trace one resource end to end before editing: root invocation, variable sources, module call, provider selection, backend and address in state. This reveals conventions that a directory listing misses. A wrapper script may select an account; CI may pass a tfvars file; a default provider may be used only for legacy resources. Preserve those facts or propose a deliberate migration of them.

7.4 Reproducibility has several layers

The engine, providers and modules have separate versions. A provider constraint specifies acceptable versions; the root's `.terraform.lock.hcl` records the selected providers and checksums. Keep that lock file in version control and review upgrades as changes. `init -upgrade` deliberately reconsiders selections within constraints; it is not a routine prerequisite for every apply.

The lock file does not lock remote module versions. Use an exact registry module version, or an immutable Git commit for a Git source, when reproducibility requires it. A branch reference can move. Provider checksums also need to cover the runner platforms you intend to support; an initial Linux setup is not proof of a macOS setup. See the dependency lock file.

The strict fixture pins make these lessons reproducible. A production root can use an upgrade policy with a supported range and a reviewed lock change; a reusable library module often needs a broader compatible requirement. The policy should say when upgrades happen and how their plans and tests are reviewed.

Exercise 6 — a helpful abstraction. An agent proposes one "universal host" module with forty optional inputs, embedded provider settings, and a module-level dependency on all networking. What concrete pressures would justify extraction, and which parts of this proposal increase review difficulty? Worked answer

Ask an agent: "Which policy does this module enforce, and which decisions remain visible to the caller?" A good answer includes the typed interface, provider expectations, ownership and migration plan. Warning signs include abstractions justified only by line count, unspecified module versions, or outputs that reveal the entire internal resource object.

8. Adopt an existing fleet without recreating it

8.1 Begin with ownership, not import commands

An existing system already has identities and writers: cloud consoles, scripts, Salt, Ansible, previous IaC, vendor automation and people. Before adoption, identify the objects, who owns which properties, where state may already exist, and what must keep working during transition.

Importing an object into a second live state does not transfer ownership away from the first. Two states can now issue contradictory changes. Likewise, importing a VM does not make its operating-system configuration an OpenTofu responsibility. The transition must retire or narrow previous writers as the new writer takes ownership.

A practical first unit is a non-critical service with an identifiable owner, documented dependencies and a reversible operational path. "Reversible" here may mean that an old configuration manager can continue serving existing hosts while a new handover path is proven. It rarely means that all infrastructure changes can be undone by switching executables.

Create a transition record for each class of object: its current API identity, the intended resource address, the current and future owner, fields being adopted, and the evidence required before the next step. This record is especially useful when adoption spans months. A plan only tells you what the current configuration proposes, not which future migration phase a machine belongs to.

8.2 Import binds an existing identity

For a provider supporting a string import ID, a declarative import has this shape:

```
# Illustration only: the resource and its real arguments
# must also be configured for the selected provider.
import {
  to = aws_instance.status["status_a"]
  id = "i-0123456789abcdef0"
}
```

This is not a runnable AWS example: it omits the provider configuration and resource definition. The point is the association between a known remote ID and a planned address. Some current providers also support structured import identities; use the selected resource's documented import format.

The corresponding resource configuration is essential. Import does not infer your intended policy. A plan may both import the object and propose changes to make it match your written arguments. Reviewing only the import count can therefore approve an immediate replacement or destructive update. The official import workflow describes the configuration requirement and provider-dependent identities.

Adopt conservatively. Observe the existing object's relevant properties through the provider API, write a configuration that represents the agreed baseline, plan the import, and account for every proposed action. For the first adoption step, aim for binding the intended object with no unintended infrastructure mutation. If the baseline violates policy, record a separate deliberate remediation rather than concealing it inside import.

After applying the adoption plan, inspect the binding, generate another plan and compare external properties with the recorded baseline. Import is evidence of management association, not evidence that the service is correctly configured or healthy.

Generated configuration can help discover schema, but it cannot decide which provider defaults are policy, which observed values are accidental, or which fields another system should own. Treat generated output as a draft requiring that interpretation.

8.3 Adopt in stages with a narrow handover

Consider a fleet currently configured by Salt. One possible sequence is:

1. Keep existing hosts running and record their identities and dependencies.
2. Put a small set of externally owned addresses behind an explicit handover schema. Validate the consumer without transferring infrastructure ownership.
3. Adopt a bounded set of API resources into OpenTofu. Keep operating-system configuration with its existing owner.
4. Make one routine non-disruptive change through the new path and prove repetition and external behaviour.
5. Move configuration management separately, host group by host group, with a defined cutover from the previous writer.

This is a design example, not an obligatory order. A provider with poor import coverage may make some resources unsuitable for early adoption. A VM without an appropriate API may remain externally provisioned while DNS or another service is managed through a provider. "Everything must become OpenTofu" is not a prerequisite for a useful system.

The manifest example gives a small contract to expand: stable host key, address and role, with a schema version. A real contract might also contain environment and inventory groups. It should exclude private keys and application credentials. Check that addresses belong to the expected objects before configuration management uses them. Publishing a manifest is not proof that SSH, privilege escalation or the service is ready.

Cloud-init or a machine image can provide a minimal bootstrap path. Salt or Ansible can own subsequent packages and service configuration. If an image and Ansible both write the same persistent setting, decide which is authoritative after boot. An infrastructure provisioner that invokes SSH scripts ties host configuration failure to infrastructure lifecycle and may run on a different runner from the one you expected. The provisioner documentation recommends other approaches where possible.

8.4 Leaving management is also a change

Deleting a managed resource block normally expresses that the object should be destroyed. To end management while retaining an object, use an explicit reviewed forgetting operation. A `removed` block can

express this intent:

```
# Illustration for an entire resource, not one keyed member.
removed {
  from = aws_instance.status
  lifecycle {
    destroy = false
  }
}
```

The resource block is removed as part of this change. In the documented 1.13 syntax, `from` cannot include an instance key: the example refers to the resource's instances as a whole. A selective handoff needs an appropriately scoped migration procedure rather than copying this block with a `for_each` key. See resource removal syntax.

OpenTofu 1.13 also has managed-resource `lifecycle.destroy = false`, which persists in state and changes removal/replacement into forgetting the old object. That differs from `prevent_destroy`, which rejects destruction while configured. Use these intentionally; a forgotten object remains alive and needs an owner, cost tracking and possibly later cleanup. See resource lifecycle behaviour.

For a transfer between live states, freeze both writers and follow a planned, backed-up transfer procedure. Prove that the receiving configuration binds the original identities and that the old writer no longer owns them. Do not allow overlapping automated runs during an improvised sequence of `state rm` and import.

Exercise 7 — adoption with improvement. A VM import plan also proposes replacing its root disk because an agent has "normalised" the image and disk arguments. The import ID is correct. Is the adoption ready? How would you structure the next steps? Worked answer

Ask an agent: "What changes ownership in this phase, and what proves that everything else continues to have one writer?" Expect current identities, old/new state ownership, field boundaries, the full adoption plan and an observable cutover. Warning signs include importing into a second live state, blanket `ignore_changes`, or using a replacement to avoid understanding an existing object.

9. Establish evidence at several levels

9.1 Each check answers a different question

Formatting checks make the HCL consistent. Validation checks configuration structure and internal consistency with available schemas. Neither establishes cloud access, quota, correct account, service reachability or safe replacement. See validate.

A normal plan adds provider observations and action predictions. Execution adds evidence that the requested API operations can complete in that environment. An external acceptance test checks the property you actually care about: a DNS answer, an application response, a restore, a management connection, or a capacity margin.

Repeat execution checks another property. A second unchanged plan should normally be empty for a stable fixture. If it is not, find out why: volatile data lookup, provider normalisation, a competing writer, a local artifact absent on this runner, or a real new change. "The first apply succeeded" does not answer any of those.

Think of verification as a set of claims with different witnesses, rather than a ladder where one green stage proves everything below and above it. A provider can successfully create a security rule permitting too much access; a health check can probe the old host instead of the replacement.

9.2 Encode contracts that should block progress

Variable validation rejects inputs that violate a module's contract. Resource preconditions can block an operation when a required condition fails. Postconditions evaluate the resource's result, so failure can be reported after an object has changed. They do not roll it back.

A `check` block is different: an assertion failure warns during normal plan/apply rather than serving as a hard gate. Do not put a must-not-violate production invariant only in a `check` and assume CI will fail automatically. Choose blocking conditions or an explicit policy/test gate according to when the evidence is available. See custom conditions and checks.

There is a timing question as well as a boolean question. A precondi-

tion based on values known at plan time can fail before changes. If its value is unknown until a dependency is created, the decision can be deferred to apply. Our failure fixture relies on exactly that timing. An assertion's placement does not mean the entire run has remained unchanged up to that point.

9.3 Read a test's execution mode

The manifest's `tests/contract.tftest.hcl` contains these tests:

```
run "handover_contract" {
  command = plan
  assert {
    condition     = output.handover.schema_version == 1
    error_message = "Consumers require handover schema version
 ↪  1."
  }
  assert {
    condition     = output.handover.hosts["status_a"].address ==
 ↪  "192.0.2.10"
    error_message = "The fixture must preserve status_a's
 ↪  address."
  }
}

run "reject_wrong_role" {
  command = plan
  variables {
    hosts = {
      db = { address = "192.0.2.20", role = "database" }
    }
  }
  expect_failures = [var.hosts]
}
```

The first asserts the consumer contract for the fixture. The second checks that our authored validation rejects an unsupported role. It is valuable because a validation that silently stops enforcing its condition is otherwise easy to miss. These tests establish neither that the file was written nor that a remote host exists.

`tofu test` defaults to applying, not merely planning, unless a run says

otherwise. Real providers can therefore create billable resources. The command uses test state and attempts cleanup, but external names and API accounts can still collide with other activity. Plan-only tests can still read provider APIs; mocks test assumptions about a provider, not the real provider's behaviour. `expect_failures` checks authored custom conditions, not arbitrary provider errors. See the test command.

9.4 A bounded acceptance record

For an agent-written change to the status handover, useful evidence would include the exact input change; an action-by-action plan for the intended state; an independent parse and schema check of the generated JSON; and a repeat plan. For a real host adoption, add observed remote identity and service checks before and after. For a replacement, add cutover and data-preservation evidence.

Ask the agent to identify what was *not* tested. A local fixture can establish engine semantics and provider-file behaviour; it cannot establish your cloud's import format or boot sequence. A well-bounded claim is more useful than a long list of commands whose relevance is unclear.

Keep the check proportionate. You do not need a miniature testing framework to rename a local output. You do need evidence of identity preservation if that output change accompanies a resource refactor, and realistic execution if a new provider upgrade changes lifecycle behaviour.

Exercise 8 — green but wrong. CI runs `fmt`, `validate` and an ordinary `apply` whose configuration contains a failing `check` assertion. The apply still succeeds. Which assumption was wrong? What additional evidence would make a host replacement acceptable? Worked answer

Ask an agent: "Which claim does each check establish, and how could this still pass while being wrong?" Expect concrete counterexamples and matching independent checks. Warning signs include treating mocked tests as provider integration, health checks without object identity, and test runs whose default apply mode went unnoticed.

10. Recover from partial completion

10.1 A failure you can predict

The complete `examples/partial-failure/main.tf` is deliberately artificial. It creates one built-in resource, then evaluates a condition that blocks a second. No cloud, files managed by an external provider, shell provisioners or credentials are involved.

```
terraform {
  required_version = "= 1.13.0"
}

variable "allow_second" {
  type    = bool
  default = false
}

resource "terraform_data" "first" {
  input = "ready"
}

resource "terraform_data" "second" {
  input = terraform_data.first.output
  lifecycle {
    precondition {
      condition = terraform_data.first.output == (
        var.allow_second ? "ready" : "blocked"
      )
      error_message = "Deliberate failure: allow_second is
 ↪  false."
    }
  }
}
```

On fresh state, `first.output` is unknown during planning. The equality cannot yet be evaluated, so the plan proposes both resources and defers the precondition. Applying creates `first`, whose output becomes `"ready"`. With `allow_second = false`, the other side is `"blocked"`; the precondition fails before `second` is created.

The important observed result is this combination: the command fails,

`first` exists in state, and `second` does not. This is more informative than either the error message or the plan summary alone.

```
tofu init
tofu plan -out=first.tfplan
tofu apply first.tfplan
# Expected error after first is created.

tofu state list
# terraform_data.first

tofu plan -var=allow_second=true -out=recovery.tfplan
# One to add; first is unchanged.
tofu show recovery.tfplan
tofu apply recovery.tfplan
tofu plan -var=allow_second=true -detailed-exitcode
# No changes; exit status 0.
```

The recovery does not reuse the failed saved plan; it produces a new plan from the new state. Its approval still needs review. Also notice that the final command repeats the corrected variable. Running later with the original default does not mean "remember the last command's flag": root inputs are resolved again.

10.2 Diagnose before repairing

For a real failed run, preserve the command, source revision, input provenance, provider versions, state identity, logs and timestamps. Establish that no runner is still active. Inspect the reported resource address and API error; then compare recorded bindings with the actual object. Be careful with log collection because provider debug logs can contain secrets.

Several superficially similar symptoms need different responses:

Symptom	Evidence to distinguish causes	Likely next decision
State lock held	Runner/job identity, process status, lock metadata	Wait for the writer or recover an abandoned lock.

Symptom	Evidence to distinguish causes	Likely next decision
Object reported absent	Account, region, principal, API lookup, state ID	Repair targeting before accepting disappearance.
Replacement after failed create	Plan reason, taint status, actual object health	Replace, or prove health before considering untaint.
API timed out after create	Provider/API logs, remote lookup, state binding	Reconcile the uncertain outcome; avoid a duplicate create.
Repeated unexpected diff	Normalisation, external writer, volatile lookup	Fix ownership or inputs rather than repeatedly applying.

An API timeout is an epistemic problem: failure to receive success is not proof that nothing happened. The remote object might exist and be recorded, exist but require adoption, or genuinely not exist. Which case holds determines recovery. Guessing from the command's exit status is inadequate.

Failed creation can mark an incomplete resource as tainted so a later plan replaces it. Not every failure taints every object. If replacement is the correct remedy, `plan -replace=ADDRESS` makes the request reviewable. Untainting is justified only after establishing the object's health; it is not a way to make a frightening plan disappear. See taint and replacement.

10.3 Drift has three possible policies

When an external actor changes a managed property, decide whether to restore the configured policy, accept the change as the new policy, or delegate that field to another owner. The engine cannot make this organisational decision.

To restore, inspect a normal plan and apply the justified correction. To accept, update the configuration and review the resulting plan. A reviewed refresh-only plan can record current observations without rec-

onciling remote objects, but it does not edit the configuration. Leaving old desired arguments in place can make the next normal plan propose undoing the accepted drift.

Prefer the reviewable `plan -refresh-only`/apply workflow to the deprecated `refresh` command, which effectively auto-approves refresh-only state changes. Incorrect targeting can otherwise make real resources appear absent in state without review. See refresh and its replacement workflow.

If a property is delegated, document the owner and scope of `ignore_changes` where appropriate. "Ignore everything" is not a repair for an unexplained diff. It removes evidence from future plans while leaving the underlying responsibility unresolved.

10.4 Recovery is not time travel

Reverting configuration asks for a new future matching earlier intentions. It may create replacements, fail to reacquire an old name, or leave changed data untouched. Restoring old state changes recorded associations; it does not restore the objects themselves. Restoring a backup changes data and may require coordination with application state and traffic.

Use a state-history restoration only as part of an investigated repair, with writers stopped and identities checked against the current world. If a failed state write leaves an emergency local snapshot, preserve it securely and follow the backend recovery procedure; rerunning from an obsolete remote snapshot can lose the only record of newly created objects. See state storage and recovery.

For a stateful service, the recovery design needs to exist before replacement: backup, restore procedure, identity/cutover policy and a tolerable data-loss window. For a stateless host, keeping the previous machine out of deletion until health is proven may be enough. These choices belong in the service design, not an improvised response to the first failed apply.

Exercise 9 — an apparently missing database. After a credential change, a plan proposes creating a database that is definitely serving traffic. An agent suggests applying because the configuration looks unchanged. Which evidence should stop that action, and what should be

inspected next? Worked answer

Ask an agent: "What do we know exists, what is uncertain, and which observation will distinguish the hypotheses?" Expect state and API identities, an explanation of partial completion, and a fresh bounded plan after repair. Warning signs include restoring state as rollback, deleting a lock immediately, or retrying until the error disappears.

11. Choose an engine and a bounded next step

This chapter is an optional branch. The previous chapters supply the model needed to evaluate it.

11.1 Shared ancestry does not prove interchangeability

OpenTofu and Terraform share a language and provider ecosystem, so learning resource addresses, dependencies, modules and state transfers well. Compatibility nevertheless belongs to particular engine, provider, module and backend versions. A tutorial that tells you merely to rename the executable is making a claim it has not established.

OpenTofu-specific features include its client-side state/plan encryption and newer language capabilities. Same-basename `.tofu` files override `.tf` files in OpenTofu; Terraform ignores `.tofu`. That can be a deliberate portability technique, but it also means the engines may read different configuration from the same directory. The newer `language` settings block is accepted by OpenTofu 1.12 onward and is not understood by Terraform. These examples retain `.tf` files and conventional `terraform` settings; they have been executed only with the stated OpenTofu target. See file precedence and compatibility settings.

When reviewing inherited work, first establish the intended engine. Check scripts, CI images and version files as well as HCL. Do not combine a provider upgrade, address refactor, backend migration and engine switch into one unexplained plan. Any of those can change behaviour independently.

11.2 Treat engine migration as a controlled change

A reasonable migration procedure starts with an understood baseline and tested backups, freezes competing writers, records exact dependency versions, initialises the intended engine, and reviews an equivalent/no-op plan. Only then make and verify a small non-critical change. Record how the previous operating path could be restored, including whether it can still read the state after the new engine has written it. See the migration guide.

Do not assume bidirectional state compatibility. The current guide for interdependent configurations says OpenTofu reads Terraform 1.x states but warns that Terraform may not reliably read states written by OpenTofu, particularly with OpenTofu-specific features. Verify the exact pair in an isolated copy before relying on a reversal. For states consumed by other configurations, migrate consumers before the states they read, working towards producers, so remaining consumers are not surprised by newly written producer state. See interdependent migrations.

This is separate from adopting an existing fleet that has no Terraform state. Import establishes management bindings; engine migration changes the program operating already-managed bindings. You may encounter both during a long transition, but they have different acceptance conditions.

11.3 Specify one useful real change

Choose one low-impact object class with an API and a clear owner. DNS for a non-critical service might fit; a production database with unknown backup behaviour probably does not. Inspect provider support for reads, import, updates and deletion before selecting the pilot.

A useful initial specification names the intended environment and identities, existing writers, fields to be adopted, allowed mutations, and acceptance evidence. For example: adopt these two existing non-critical records without changing their answers, retain the same external identities, prove an empty repeat plan, then make one separately reviewed TTL change. The provider's actual resource and import semantics must support that specification.

The agent can research the selected provider, produce HCL, run iso-

lated checks and present the adoption plan. Your retained decision is whether the evidence supports the ownership change and whether its failure modes are acceptable. This is where the small model becomes operational judgement: knowing what an apparently simple edit is allowed to change.

Exercise 10 — three changes at once. A proposal switches Terraform to OpenTofu, upgrades the provider and extracts a module. Its plan contains unexpected replacements. How would you separate the causes, and what would constitute an acceptable first step? Worked answer

Ask an agent: "Which compatibility and lifecycle claims depend on the exact versions, and what isolated evidence supports them?" Expect versioned sources, a baseline, state-read/write testing and a controlled plan. Warning signs include blanket compatibility assurances or restoring an older state file to make the old executable start.

Worked answers

1. The failed run

Back to exercise 1

The command's failure does not undo successful provider operations. The network and VM A may exist and be recorded; VM B's creation may have failed before creating anything, or may have left an incomplete object. Restoring old state can discard the only recorded bindings to successful new objects. A retry could then create duplicates or conflict with them.

First preserve logs and state history, establish that the writer has stopped, identify the error and its resource address, and inspect the relevant API identities. Verify account and region as well as names. Fix the cause, then generate and review a new plan. In the simple quota case with correctly recorded successes, it should retain the network and VM A and create only VM B. Different observed evidence can justify a different recovery; the exit code alone cannot.

2. The handover

Back to exercise 2

The apply establishes that a local provider wrote the configured file on its runner. It does not establish any real host allocation: the fixture's addresses were literal inputs. Even in a real configuration, a returned address needs to be tied to the intended remote identity and environment.

A real contract needs a documented schema and consumer behaviour, stable host keys, correct target provenance and an authorised publication path. Before changing a host, the consumer needs appropriate identity and access checks. After configuration, use an independent service probe and whatever acceptance criteria matter for that service. Do not put private keys into a convenient output merely because it makes the next automation step easier.

The output value and the file content agree by construction, so comparing them is not independent proof that a VM exists. It is still useful to parse the file: that checks a different implementation boundary, serialisation and publication.

3. The saved plan

Back to exercise 3

The saved artifact contains its planning decisions; editing HCL afterwards does not make that artifact a new production plan. The command does not perform the fresh plan that bare `apply` would perform. This is exactly why the provenance of the artifact matters.

That does not make arbitrary apply-time environments safe. The runner still needs the intended backend and valid provider credentials; a mismatched account can cause failures or provider-specific hazards. External infrastructure or service conditions can also change without making this state revision stale. A long delay or intervention should trigger a new review.

The acceptance record should therefore tie the plan to its source, inputs, state and effective target, and keep the execution environment controlled. The absence of a confirmation prompt when applying the artifact is another reason to make that boundary explicit.

4. The renamed key

Back to exercise 4

`status_a` is part of the resource address. Replacing it with `status_primary` changes identity unless the old/new relationship is recorded. Without a move, the engine can plan deletion of one instance and creation of another. Changing the key alone is not a display-name edit.

`create_before_destroy` orders a replacement; it neither turns it into a rename nor checks application health. It also needs coexistence capacity and compatible naming. To accept a real rename, inspect the existing remote ID, add an appropriate move from the old keyed address to the new one, and require a plan preserving the identity without unintended API changes. Prove this before mixing in an image, disk or network change.

If the change is only for presentation, retaining the stable key and introducing a separate display label may avoid the identity migration entirely.

5. The three workspaces

Back to exercise 5

There are separate state instances, which prevents one workspace's state from simply being the same set of bindings as another's. There is no demonstrated separation of credentials, API permissions, remote names or access to backend data. A powerful shared principal may still be able to change every environment.

Decide which boundaries the projects require: account/project separation, distinct principals, backend access, independent approval and change paths, and object-name uniqueness. Separate roots/backends with shared modules are often a clearer expression of materially different permissions. Workspaces remain useful when the requirement is genuinely "several similar instances under the same operational policy".

The design must also account for selecting the wrong workspace. A convenient selector is not evidence of the selected target.

6. The universal module

Back to exercise 6

A useful extraction has a stable domain boundary and policy shared by real callers: for example, a standard host attachment and a versioned handover contract. It makes the common case easier to judge and explains supported variation.

Forty pass-through inputs may expose most of the provider without simplifying its semantics. Embedded provider decisions hide the target and make reuse across accounts harder. A dependency on all networking obscures the actual values needed and can make unrelated changes affect planning.

Ask for two concrete callers and their common policy, the minimal typed interface, provider configurations supplied by each caller, and resource-address moves for existing callers. Keeping the current root until that policy is understood is a reasonable result. Fewer lines are not worth a harder ownership model.

7. Import plus replacement

Back to exercise 7

The correct import ID establishes only the intended association. The additional disk/image changes are a separate operational proposal with potentially destructive effects. “Normalise” is not an adequate reason to accept replacement of a running machine.

Establish the real baseline and provider semantics. Remove accidental desired changes from the adoption step, or explicitly explain why an adoption without mutation is impossible. Review and apply the association only when every action is acceptable. Then propose remediation separately, with backup, service continuity and data-preservation requirements as appropriate.

Do not mask the uncertainty with broad `ignore_changes`. If a particular field belongs to another system, document that ownership; if it belongs to OpenTofu, decide the desired policy and the safe transition to it.

8. The warning that did not gate

Back to exercise 8

A failing ordinary `check` assertion is a warning, so the mistaken assumption was that it necessarily blocks plan or apply. A deliberately authored test can assert that a check should fail, but that is different from making a normal deployment fail on that warning. Put a hard invariant in a suitable blocking condition or explicit external gate and test that gate's failure behaviour.

For host replacement, formatting and validation are useful but insufficient. Review identities and replacement order, verify coexistence capacity, and prove the new host's configuration and service health before cutover/deletion according to the rollout design. Check that the health probe actually reaches the new host. A repeat plan then checks reconciliation stability, not user-visible availability by itself.

9. The missing database

Back to exercise 9

The running service is evidence that an object exists, so a create proposal after a credential change is a targeting/observation discrepancy to investigate. Stop before applying. Compare the backend and state binding, provider principal, account/project, region and API identity with the known database.

Wrong credentials, a different endpoint, missing permissions or the wrong state can explain the discrepancy. Determine whether the provider reported absence, failed to read, or was looking elsewhere. Do not accept a refresh-only state change until the observation is understood either: forgetting a real database is not a safe consequence of fixing a credential profile.

Once the target is repaired, obtain a new plan. The acceptable result depends on the real baseline; "configuration unchanged" was never proof that the command addressed the same world.

10. The compound migration

Back to exercise 10

First reproduce and understand the old engine's baseline using its recorded provider/module versions and intended backend. Investigate existing unexpected changes before blaming the migration. Then test the engine switch with dependencies and resource addresses held constant, using a secure isolated state copy and an appropriate read/plan procedure. The desired first result is equivalent ownership and a no-unintended-change plan, with state compatibility and recovery understood.

Handle the provider upgrade separately, inspecting its version-specific changes and plans. Handle module extraction with explicit address moves and a plan demonstrating identity preservation. If replacements appear only after one step, there is now a much smaller set of causal explanations.

An isolated state copy does not authorise applying against the same live objects from a second writer. Isolation of state storage is not isolation of the managed API objects.

A compact operational reference

Question	Useful evidence
Which system will this command touch?	Root, backend/state, workspace, inputs, provider identity and region/endpoint.
Is this an update or a new identity?	Full resource address, remote ID, move history and provider replacement reason.
Is the approved proposal the one being run?	Saved-plan provenance and controlled execution environment.
Can another writer interfere?	State lock support, writer policy and field ownership outside this state.
What does success establish?	API outcome, external acceptance checks, correct object identity and repeat plan.

Question	Useful evidence
What survives failure?	Current state bindings, actual API objects, logs and preserved recovery data.
Can this change be reversed?	A concrete compensating change, cutover plan or tested data restore.

For `plan -detailed-exitcode`, status **0** means no changes, **1** means an error, and **2** means the plan has changes. CI must handle the latter deliberately; it is not an ordinary failure. Conversely, status 0 is not proof of correct account selection or application health. See plan exit codes.

Useful read-oriented commands include `tofu state list` for addresses, `tofu state show ADDRESS` for a particular recorded resource, `tofu show reviewed.tfplan` for a saved plan, and `tofu providers` for provider requirements. Their output may be sensitive. Raw state and JSON plan exports are not safe to publish merely because the human-readable plan used redaction.

Treat `state rm`, `state push`, `force-unlock`, `-replace` and destroy planning as operations with distinct intentions. They are not interchangeable ways to make a diff disappear. Review the hypothesis and evidence behind the chosen operation.

Reproduce the laboratories

Files and prerequisites

The labs are independent roots. Use a disposable copy of each example directory so that its state and generated files stay separate from any real project. Run each command from the corresponding lab directory. All assume OpenTofu **1.13.0** on Linux; use the official installation instructions for the appropriate platform and verify `tofu version`.

The supporting entry points are:

- Manifest configuration, provider lock file, and contract tests.

- Lifecycle configuration, role-update inputs, and replacement inputs.
- Partial-failure configuration.

The built-in-resource labs need no provider download. The manifest requires network access for the pinned local provider on first initialisation; retaining an installed provider or configuring a supported mirror is a separate preparation step. Reading this document does not depend on that installation.

The manifest writes only beneath its own `generated/` directory. Its documentation addresses are literal examples; it does not contact them. The two other labs manage only built-in-resource state. No lab needs a cloud account or root privilege. API-based examples in the chapters are labelled illustrations and are not complete cloud configurations.

Manifest: create, repeat, then detect drift

The complete configuration is in chapter 3. In a fresh disposable copy:

```
tofu init
tofu fmt -check
tofu validate
tofu test
tofu plan -out=reviewed.tfplan
tofu show reviewed.tfplan
tofu apply reviewed.tfplan
tofu plan -detailed-exitcode
```

Expect one creation, then no changes. Read `generated/handover.json` with any JSON parser and confirm schema version 1 and the two host addresses. `tofu test` uses the supplied plan-only tests and should report two passing tests, including the expected rejection case.

Now replace the generated file's content with any visibly different text using an editor. Plan again. For the pinned local provider, the tracked content is now absent, so the plan proposes creation to restore it. Apply a newly reviewed saved plan, inspect the repaired JSON, and obtain an unchanged repeat plan. This is the measured behaviour of the fixture, not a general rule that all providers represent drift as a create.

Lifecycle: update versus replace

Here is the complete initial `main.tf`:

```
terraform {
  required_version = "= 1.13.0"
}

variable "hosts" {
  type = map(object({
    role     = string
    revision = number
  }))
  default = {
    status_a = { role = "status", revision = 1 }
    status_b = { role = "status", revision = 1 }
  }
}

resource "terraform_data" "host" {
  for_each         = var.hosts
  input            = each.value.role
  triggers_replace = each.value.revision
}

output "ids" {
  value = { for key, host in terraform_data.host : key =>
 ↪  host.id }
}
```

Initialise, plan, inspect and apply the baseline as in the manifest lab. Record the two output IDs. `update.tfvars` contains:

```
hosts = {
  status_a = { role = "web", revision = 1 }
  status_b = { role = "status", revision = 1 }
}
```

`replace.tfvars` contains:

```
hosts = {
  status_a = { role = "status", revision = 2 }
  status_b = { role = "status", revision = 1 }
}
```

From the baseline state, inspect both alternative plans:

```
tofu plan -var-file=update.tfvars
tofu plan -var-file=replace.tfvars
```

The first proposes one in-place update, the second one replacement. Neither speculative plan changes the baseline. If you choose to apply a variant, save and inspect its plan first; keep supplying its inputs on subsequent plans. The other variant is then being compared with a changed baseline.

To explore refactoring independently, start from the baseline. Rename the resource block from `host` to `node`, update the output's reference to `terraform_data.node`, and add the chapter 5 `moved` block. The plan should preserve both IDs and show no infrastructure creation or deletion. Without the move, the same spelling change represents different addresses.

Failure: recover from what remains

Use a fresh copy of the complete chapter 10 fixture. Follow its command sequence. Expect an apply error after `first` is created, then a state listing containing only `terraform_data.first`. The corrected plan with `allow_second=true` should propose only the missing second resource; a repeat plan with the same flag should be empty.

Once each lab is finished, `tofu destroy` in that disposable root removes its managed fixture resources. In the failure lab, supply `-var=allow_second=true` to retain the corrected input consistently. Inspect the destroy plan and use the interactive approval. Local state and plan artifacts remain ordinary files; retain or remove the disposable copy according to your experiment needs. Do not use a cleanup command against an unrelated working directory.

Verification boundary

The local examples were executed with OpenTofu 1.13.0 on Linux amd64. Checks covered initial apply, no-change repetition, the two lifecycle variants, a move preserving identity, external file-edit repair, both contract tests, and deliberate partial failure followed by recovery. The fixture plans were inspected as structured data as well as read in human form. The main forms of chapter 1 were checked on 3 November 2026 with the same engine, and `verification.md` lists them.

No real cloud resource, backend migration, state encryption/key recovery, live fleet import, SSH bootstrap or Terraform engine migration was executed. Those sections explain documented semantics and design choices; applying them to a particular provider needs that provider's versioned documentation and an isolated acceptance test. The companion verification record identifies the executed checks and limitations.